Deliverable 2 - Campus Coffee Shop Sales Pipeline

In [106…

```python
import json
import re
from pathlib import Path

import numpy as np
import pandas as pd
```

Link to the repository

https://github.com/EllenAlnes/deliverable_2_coffe_shop.git

Link to video

https://nmbu.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=c78eabfe-1eb2-478b-ae22-b4f200b2991f

Task 1 File ingestion and regular expressions

Function `read_file()` uses the Path to read a file, it is written to handel utf-8 or ISO-8859-1/Latin-1 encoding. It returns a tuple containing the files content and the used encoding as strings.

In [107…

```python
def read_file(input_file: Path) -> tuple[str, str]:
    """Read a file and return its content and the encoding used.

    Arguments:
        input_file: Path to the file to read.

    Returns:
        A tuple containing the file content as a string and the encoding used as a st

    try:
        # Try to read the file with UTF-8 encoding
        with input_file.open(encoding="utf-8") as file:
            file_text_content = file.read()
            encoding_used = "utf-8"

    except UnicodeDecodeError:
        # If UTF-8 fails, try ISO-8859-1/Latin-1 encoding
        with input_file.open(encoding="ISO-8859-1") as file:
            file_text_content = file.read()
            encoding_used = "ISO-8859-1"

    # Returns a tuple with file_text_content and encoding_used
    return file_text_content, encoding_used
```

The function `parse_log()` parses the log file contents and returns the valid transactions as a list of dictionaries. The functions uses regular expresions to filter out the valid transactions. The regex pattern groups the different elements together and names each group. The different groups are

then organised as dictionary keys. The filterd out groups are: timestamp, register_id, item, price, currency and status.

In [108…

```python
def parse_log(file_text_content: str) -> list[dict[str, str | float]]:
    """Parse log file content and and return valid transactions.

    A parser function analyses text and extracts structured information from it accor

    Arguments:
        file_text_content: The complete text content of a log file.

    Returns:
        A list of dictionaries containing valid transactions.
        Each dictionary contains the timestamt, register ID, item,
        price, currency, and status. The price is stored as a float.
        Lines that do not match the expected log format are ignored.
    """

    # Regex pattern for valid transactions
    # () - groups result together
    # ?P<...> names the group
    # r"..." raw string
    pattern = (
    r"\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] "
    r"\[(?P<register_id>\w+)\] "
    r"item=(?P<item>\S+) "
    r"price=(?P<price>\d+\.\d+) "
    r"(?P<currency>EUR|NOK|USD) "
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)"
)
    # empty list to collect the valid lines in the logfiles
    valid_transactions = []

    # splits up the file so we can work with it line by line
    for line in file_text_content.splitlines():

        # check whether this line matches the pattern
        match = re.fullmatch(pattern, line)

        # if the line is valid, extract this field
        if match:
            transaction = match.groupdict()

            # convert prise to float
            transaction["price"] = float(transaction["price"])

            # add the transaction to the list
            valid_transactions.append(transaction)

    return valid_transactions
```

Task 2 recursive utility functions

The code `find_log_files()` is used to find .log files in a directory. It takes in a path to the startingpoint of the directory. Then it recursively searches down the path until it finds the .log files. It returns a list of path objects representing the log files it finds.

In [109…

```python
def find_log_files(directory: Path) -> list[Path]:
    # directory: Path - funksjonen mottar en mappebane
    # -> list[Path] - funksjonen skal returnere en liste med filbaner - ikke filinnho
```

```
    """Find all log files in a directory and its subdirectories.

    The function searches recursively through the directory structure
    and collects the paths of all files ending in .log.

    Arguments:
        directory: Path to the directory where the search begins.

    Returns:
        A list of Path objects representing the log files found.
    """

    all_log_files = []

    for item in directory.iterdir():

        if item.is_dir():
            # Search the subdirectory recursively and collect its log files

            all_log_files.extend(find_log_files(item))  # This calls the function aga

        elif item.is_file():
            # check file ending - .suffix gives fileending including .
            if item.suffix == ".log":
                all_log_files.append(item)

    return all_log_files
```

The function `find_active_registers()` searches nested dictionaries and lists to find active register ids. This function is used on a the store_config.json file, witch gives information about the configuration of some of the stores in the company.

In [110…

```
def find_active_registers(data: dict | list) -> set[str]:
    """Find IDs fo all active registers in nested data.

    Arguments:
        data: nested dictionaries and lists to search.

    Returns:
        A set of IDs belonging to active registers"""

    # empty set for collecting active registers
    active_registers = set()

    if isinstance(data, dict):
    # isinstance(item, dict) asks if the item a dictionary
        # Check wheter this dictionary describes an active register
        if (
            "id" in data
            and data.get("active") is True
        ):
            active_registers.add(data["id"])

        # Search recursively through the dictionary values
        for value in data.values():
            # data.values() gives the values inside the dict
            # each value gets sendt back to the same function
            # if it is a dict og a list the function "searches" it
            # .update() adds all the found ids into the existing set
            active_registers.update(find_active_registers(value))

    elif isinstance(data, list):
```

```
    # isinstance(item, list) asks if the item a list
        # Search recursively through each element in the list
        for item in data:
            active_registers.update(find_active_registers(item))

    return active_registers
```

Task 3 - Cleaning and analysing

The cleaning function changes outlying 999 transactions values in the dataframe, to the meadian of valid transactions. This is easily done with mathematical operations on numpy arrays. The output is then an array of all cleaned prices.

In [111…

```
# make cleaning function:
def clean_outliers(df: pd.DataFrame) -> np.array:
    """Changes outlying transactions values in the dataframe, to the meadian of valid

        Arguments:
            df: DataFrame with invalid outlying price values.

        Returns:
            Array containing outlier-cleaned transactions.
        """
    # converts price values to array
    cleaned = np.array(df["price"])

    mu = np.mean(cleaned)
    sigma = np.std(cleaned)

    # gather outlying values
    outliers = cleaned > mu + 3*sigma

    valid_prices = cleaned[~outliers]
    median_valid = np.median(valid_prices)

    # replaces all outlier values in cleaned, with the median of the valid prices.
    cleaned[outliers] = median_valid

    return cleaned
```

The next function calculates the ratio between element-wise refund and sales values. With sales and refund values as input, this will be used later on, to make an array of ratios in task 4.

In [112…

```
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.nda
    """Calculates the ratio between element-wise sales and refund values.

        Arguments:
            sales_count: Array of the sales count for each element.

            refund_count: Array of the refund count for each element.

        Returns:
            Array containing the refund/sales ratio element-wise.
        """
    ratios = []
    for ref, sale in zip(refund_count, sales_count):
        rat = ref / (sale + 1e-6)
        ratios.append(rat)
    ratios_arr = np.array(ratios)
```

```
    return ratios_arr
```

Task 4 - Data analysis using Pandas

The `chrono` function sets the column [timestamp] to pd.datetime so Python handles it as time, and sorts it chronologically.

In [113…
```
def chrono(df: pd.DataFrame) -> pd.DataFrame:
    """Creates DataFrame of a list of dictionaries. Sorts the transactions by timesta

    Arguments:
        df: DataFrame containing information of transactions.

    Returns:
        DataFrame with transactions in chronological order.
    """
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp").sort_values("timestamp")
    return df
```

The next functions returns the total prize for each product within 1-hour windows, pivot tables sorting item revenue across status "SUCCESS", "REFUND" and "CANCELLED", and calculates the ratio between element-wise refund and sales values in given DataFrame. This last function makes use of the `compute_refund_ratio` for this specific datafram.

In [114…
```
def one_hour(a_dataframe: pd.DataFrame) -> pd.DataFrame:
    """Calculates total price for each product within 1-hour windows.

    Arguments:
        a_dataframe: dataframe sorted by timestamp, containing transactions.

    Returns:
        DataFrame sorted by 1-hour windows. One row for each item, within each 1-hour
    """
    return a_dataframe.groupby("item").resample("1h")["price"].sum()
```

In [115…
```
def by_status(b_dataframe: pd.DataFrame) -> pd.pivot_table:
    """Creates pivot table, sorting item revenue across status "SUCCESS", "REFUND" an

    Arguments:
        b_dataframe: dataframe sorted by timestamp, containing transactions.

    Returns:
        Pivot table.
    """
    return pd.pivot_table(
        b_dataframe,
        index="item",
        columns="status",
        values="price",
        aggfunc="sum",
        fill_value=0
    )
```

In [116…
```
def use_ratio(c_dataframe: pd.DataFrame) -> np.array:
    """Calculates the ratio between element-wise sales and refund values in given Dat

        Arguments:
```

```
            c_dataframe: A dataframe containing transactions, either a sale, refund o

        Returns:
            Array containing the sales/refund ratio element-wise.
        """
    counts = c_dataframe.groupby(["item", "status"]).size().unstack(fill_value=0)
    success_arr = counts["SUCCESS"].to_numpy()
    refund_arr = counts["REFUND"].to_numpy()

    return compute_refund_ratio(refund_arr, success_arr)
```

Task 5 Full workflow

Recursively find all log files and extract active register IDs

Here we find all of the log files starting with the path "data/2026". We use the recursive function to go through the directory.

In [117…
```
data_dictionary = Path("data/2026")
log_files = find_log_files(data_dictionary)

print(f"Number of log files found: {len(log_files)}")
```

```
Number of log files found: 9
```

Here we find the active register ids from the store config file. Since the file did not have information about all of the stores we chose to just do the analysis about the active register and extracting the information, but we did not use it to filter out any of the data from the transaction logs.

In [118…
```
# Store confuguration - data from store-config.json
config_path = Path("data/store_config.json")

config_text, encoding_used = read_file(config_path)
# json.loads konverts text into python datastructure with dictionaries and lists
# loads - string of text, while load takes a fileobject
config_data = json.loads(config_text)

# Find all active register IDs
active_registers = find_active_registers(config_data)

# Print register information by store in config file
for region in config_data["regions"]:
    for store in region["stores"]:
        print(store["name"])

        for register in store["registers"]:
            if register["active"]:
                status = "Active"
            else:
                status = "Inactive"

            print(f"{register["id"]}, {status}")

# sorted sorts the set alfabeticly
print(f"Active register IDs form config: {sorted(active_registers)}")
```

```
Library Cafe
POS_01, Active
POS_02, Active
Science Building
POS_03, Active
POS_04, Inactive
Active register IDs form config: ['POS_01', 'POS_02', 'POS_03']
```

Ingest text logs and parse records into a list of dictionaries

In this step, we process each log file individually and collect all valid transactions into a single list. First, we create an empty list called `all_transactions`. We then loop through the log files in sorted order to ensure a consistent processing order.

For each file, `read_file()` reads the text content and handles the file encoding. The content is then passed to `parse_log()`, which extracts valid transaction records and returns them as a list of dictionaries. Each dictionary represents one transaction and contains the relevant information, such as the timestamp, register ID, item, price, currency, and status.

The `extend()` method adds all transactions from the current file to `all_transactions`. This allows us to combine the records from all nine log files into one list for further processing. Finally, we print the total number of transactions collected. The result is 720 transactions.

In [119…

```python
# Create empty list for all transactions
all_transactions = []

# Read and parse each log file
# Collect all transactions into one combined list
for log_file in sorted(log_files):
    file_text_content, encoding_used = read_file(log_file)
    transactions = parse_log(file_text_content)
    all_transactions.extend(transactions)

print(f"Number of transactions: {len(all_transactions)}")
```

```
Number of transactions: 720
```

Build the initial Pandas DataFrame

In this step, we convert the list of transaction dictionaries into a Pandas DataFrame. We then inspect the first rows, check the data types and missing values, verify the total number of transactions, and count the transactions from each register.

In [120…

```python
# Task 5.3 Build the initial DataFrame
df = pd.DataFrame(all_transactions)

# Check the DataFrame
print(df.head())
df.info()    # Gives info about the DataFrame
print(f"Number of transactions: {len(df)}")
print(df["register_id"].value_counts())
```

```
             timestamp register_id        item  price currency     status
0  2026-10-15 07:02:53      POS_01  Cappuccino   4.44      EUR  CANCELLED
1  2026-10-15 07:05:22      POS_01    Espresso   2.73      EUR     REFUND
2  2026-10-15 07:05:38      POS_01    Sandwich   7.42      USD    SUCCESS
3  2026-10-15 07:08:18      POS_01    Espresso   2.84      EUR    SUCCESS
4  2026-10-15 07:09:23      POS_01  Cappuccino   3.80      EUR    SUCCESS
<class 'pandas.DataFrame'>
RangeIndex: 720 entries, 0 to 719
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   timestamp    720 non-null    str
 1   register_id  720 non-null    str
 2   item         720 non-null    str
 3   price        720 non-null    float64
 4   currency     720 non-null    str
 5   status       720 non-null    str
dtypes: float64(1), str(5)
memory usage: 33.9 KB
Number of transactions: 720
register_id
POS_01    240
POS_02    240
POS_03    240
Name: count, dtype: int64
```

The code for task 5-4 to 5-7 makes use of the small functions in task 3 and 4 for making the CVS, JSON and pivot outputs attached.

Apply NumPy outlier cleaning to remove 999.00 price glitches

In [121…

```
# the column [price] will be changed in the dataframe df, to the cleaned values-array
df["price"] = clean_outliers(df)
```

Apply currency correction: multiply price by 10 for all transactions where currency == "NOK"

In [122…

```
df.loc[df["currency"] == "NOK", ["price"]] *= 10
df.to_csv("cleaned_transactions.csv")
```

Generate resampled reports, pivot tables and refund ratios

In [123…

```
chronologically = chrono(df)
hourly_revenue = one_hour(chronologically)
hourly_revenue.to_json("hourly_revenue.json") # A JSON report containing total revenu

revenue_pivot = by_status(df)
revenue_pivot.to_csv("status_pivot_table.csv") # A CSV file containing a pivot table

rat_items = pd.DataFrame(use_ratio(df))
rat_items.to_csv("rat_items.csv", index=False) # A CSV table containing refund ratios
```